

O-notation for Algorithms

When we talk about the *running time* of algorithms (or sometimes, the memory usage) in a theoretical setting, we typically use functions to describe the running time, or how much *work* the algorithm does on an input of size n .

Linked List Search

Consider a linked-list with n numbers. If we write a function to determine if, say, the number 5 is in the list, then we could say:

- at *best*, the 5 is the first element and we only need to look at one node
- at *worst*, the 5 is the last node, or not in the list at all, and we need to check all n nodes
- on *average*, somewhere between 1 and n nodes must be checked to find the 5

For the moment, think about the worst case only. We can describe its running time by the function $f(n) = n$. This means that given a linked list of size n , the worst case running time looks like the function $f(n) = n$.

Another way to describe $f(n) = n$ is to say that the running time of this algorithm is *proportional* to n . As n gets bigger, the running time gets proportionally bigger.

For example, what is the running time of searching for the number 5 in a linked list of size 5000? Since $f(n) = n$ is the running time, the answer is $f(5000) = 5000$.

Note that there are *no* units: we *cannot* say if this means microseconds, or minutes, or days, etc. All we know is that the running time is proportional to n , i.e. that it looks like the function $f(n) = n$ when graphed.

Despite this vagueness it does let us answer some useful questions. For example, if it takes 1 second to search for the number 5 in a linked list of size 5000, how long will it take to search for the number 5 in a linked list of size 10,000? 10,000 is twice the size of 5000? Since $f(n) = n$, we know that $f(2n) = 2n$. So doubling the size doubles the running time. So if it takes 1 second to search 5000 nodes, it will take about 2 seconds to search 10,000 nodes.

Insertion Sort

Recall that **insertion sort** is a sorting algorithm that re-arranges the elements of a list into ascending order. Conceptually it works like this:

- Assume the list is called **unsorted**
- Make a new empty list called **sorted**
- Repeat the following until **unsorted** is empty:

- Remove any element from **unsorted**, and call it **key**
- Insert **key** into the correct position in **sorted**. This can be done by searching through **sorted** for the correct position, moving elements up as needed.

In general, we can describe the running time of insertion sort by the function $f(n) = n^2$. This means that given a list of size n , the worst case running time of insertion sort is n^2 .

Again, there are *no* units. All we know is that if we graph the running time it will be a parabola. We say that the running time of insertion sort is *proportional* to n^2 .

Here's an important observation: if you *double* the input size of insertion sort, the running time increases by a *factor of 4* (not 2!). This is because $f(2n) = (2n)^2 = 4n^2 = 4f(n)$.

For example, suppose it takes 3 seconds to sort 50,000 items with insertion sort. About how long to do you it expect it to take to sort 100,000 items (twice the size)? The running time should increase by a factor of 4, and so we'd expect it to take about $4 \times 3 = 12$ seconds to sort 100,000 items.

Combining Running Times

Suppose you have an array of n numbers and do the following:

- You search the array for the number 5 using a linear search. The running time of this is, in the worst case, described by the function $f(n) = n$.
- You then sort the array using insertion sort. The running time of this is, in the worst case, described by the function $g(n) = n^2$.

What is the total running time of these two operations? Clearly, it is $f(n) + g(n) = n + n^2$. If you graph $n + n^2$, you get a parabola. This is an important observation: the total running time of these two operations is quadratic, i.e. it is proportional to n^2 .

Other Common Running Times

Constant Time

Suppose you have a linked list of n numbers that you know is in sorted order from smallest to largest. What function describes the running time of the best algorithm for finding the minimum value in the list?

Since the list sorted, the *first* value is the minimum, so it takes only one operation, i.e. $f(n) = 1$. This is a **constant time operation**, which means that the time it takes to run does *not* depend on n . No matter how big or small the list is, it will never take more than some fixed constant amount of time to find the minimum value.

Logarithmic Time

Suppose you have a sorted array of n numbers and you want to find if the number 5 is in it. A fast way to do this is to use **binary search**: check to see if the middle element is 5, and, if it's not, repeat on either the left or right half of the array (depending on whether the middle element is greater or less than 5). After each comparison the size of the array is halved, and so in the worst case it ends up doing about $\log_2 n$ comparisons.

This means the function describing the running time of binary search is $f(n) = \log_2 n$.

$\log_2 n$ is a **logarithmic function**, and it shows up all the time in algorithm analysis. Make sure you memorize the basic rules for how logarithms work: see the textbook for a refresher.

Notice what happens to the running time when you double the input size: $f(2n) = \log_2(2n) = \log_2 2 + \log_2 n = 1 + \log_2 n = 1 + f(n)$. So doubling the input size increases the running time by 1, which is a small increase.

Linear Time

The linked list search example from above is an example of a **linear time** algorithm. A linear time algorithm is one whose running time is proportional to the input size, i.e. it is described by the function $f(n) = n$. More generally, it could be of the form $f(n) = an + b$, where a and b are constants.

$n \log n$ Time

Another common running time for algorithms is $n \log n$. For example, the **mergesort** sorting algorithm (which we will see later) has a running time of $f(n) = n \log n$.

Suppose you have an array of n numbers and you do these two operations:

- Sort it with mergesort. The running time of this is $f(n) = n \log n$.
- Search it with binary search. The running time of this is $g(n) = \log n$. (In computer, we don't usually bother to write the base of the logarithm when it is 2.)

The total running time is $f(n) + g(n) = n \log n + \log n = (n + 1) \log n$. In other words, the total is very nearly $n \log n$. The time to do the sorting dominates the time to do the searching.

Cubic Time

Consider the problem of multiplying together two $n \times n$ matrices (as in linear algebra). The standard algorithm for doing this by hand takes $f(n) = n^3$ operations.

If you double the size of the matrices, what is the running time? It is $f(2n) = (2n)^3 = 8n^3 = 8f(n)$. So doubling the size of the matrices increases the running time by a factor of 8 — which is a lot!

Exponential Time

Consider the problem of printing all bit strings of length n . There are 2^n such strings, and so the running time is $f(n) = 2^n$. This is an *exponential* function, and if your algorithm takes an exponential amount of time, it is a **slow** algorithm.

Important: refresh your memory on the rules for manipulating exponents: see the textbook for details!

What happens to an exponential time algorithm when you increase the input size by 1? Since $f(n) = 2^n$, we have $f(n+1) = 2^{n+1} = 2 \cdot 2^n = 2f(n)$. So increasing the input size by 1 *doubles* the running time.

What if you add 10 more items? Then $f(n+10) = 2^{n+10} = 2^{10} \cdot 2^n = 1024 \cdot 2^n = 1024f(n)$. So adding 10 more items means the algorithm runs more than 1000 times as long.

If you have 10 times as many bits, the running time is $f(10n) = 2^{10n} = 1024 \cdot 2^n = 1024f(n)$, 1024 times as long as printing n -bit strings. This is pretty bad!

Note that if you *double* the input size, the running time is $f(2n) = 2^{2n} = (2^n)^2 = f(n)^2$. This means that however long it took you to process n items, if the algorithm runs in exponential time, it will take the *square* of that running time to process $2n$ items.

In practice, exponential time algorithms are very slow and typically impractical except for small values of n .

O-notation

O-notation, or **big-O notation**, is a common way to describe the running time of algorithms (and sometimes, memory usage) in theoretical computer science.

The intuition is as follows. Suppose you have an algorithm that runs in $f(n) = n^2 + n + 5$ time. When n is big, the n^2 term dominates the other terms, i.e. when n is big, n^2 is so much bigger than the other terms that they can be ignored. And so we can say that $f(n)$ behaves like n^2 when n is big. Or, using O-notation, we can say that $f(n)$ is $O(n^2)$.

Sometimes it's useful to think about O-notation as a kind of “less than or equal to” relation. Saying that $f(n)$ is $O(n^2)$ means, intuitively, that $f(n)$ is at most n^2 , i.e. that $f(n)$ is less than, or equal to, n^2 when n is big.

We often skip the $f(n)$ function when using O-notation for algorithms. For example:

- Binary search is $O(\log n)$ runs in logarithmic time in the worst case.
- Linear search is $O(n)$ runs in linear time in the worst case.
- Quicksort is $O(n \log n)$ runs in $O(n \log n)$ (linearithmic) time in the average case.
- Insertion sort is $O(n^2)$ runs in quadratic time in the worst case.
- Matrix multiplication is $O(n^3)$ runs in cubic time in the worst case.
- Printing all n -bit strings is $O(2^n)$ runs in exponential time.

The Definition of O-notation

While the intuition for O-notation is fairly clear, it is also useful to have a precise mathematical definition. Here is the textbook's definition of what $f(n)$ is $O(g(n))$ means:

Definition of O-notation. $f(n)$ is order $g(n)$, written $f(n)$ is $O(g(n))$, if there exists a real constant $c > 0$, and an integer constant $n_0 \geq 1$ such that $f(n) \leq cg(n)$ for all $n \geq n_0$.

IMPORTANT: Memorize this definition!

This definition works a *little* bit like limits in calculus. The idea is that, when n is big, $g(n)$ is a kind of upper bound on $f(n)$.

Example. Using the definition of O-notation, prove that $3n + 5$ is $O(n)$.

Intuitively, we can see that $3n + 5$ is $O(n)$ because the high-order term of $3n + 5$ is n . More precisely, we reason as follows.

According to the definition, if $3n + 5$ is $O(n)$ then we must be able to find a real constant $c > 0$ and an integer constant $n_0 \geq 1$ such that:

$$3n + 5 \leq cn, \text{ for all } n \geq n_0$$

By inspecting the inequality, we can make a guess: if we set, say, $c = 4$ then the inequality is true then we get this:

$$3n + 5 \leq 4n, \text{ for all } n \geq n_0$$

Simplifying the inequality, we get:

$$5 \leq n, \text{ for all } n \geq n_0$$

Clearly, if we set $n_0 = 5$ then the inequality is true:

$$5 \leq n, \text{ for all } n \geq 5$$

This satisfies the the definition, and so we've proven that $3n + 5$ is $O(n)$.

We *don't* need to find the smallest values of c and n_0 . *Any* values that satisfy the inequality will do. We could have chosen $c = 40000$ and $n_0 = 1000000$.

Example. Using the definition of O-notation, prove that n^2 is *not* $O(n)$.

To do this, lets assume that it was true that n^2 is $O(n)$ and derive a contradiction. If n^2 were $O(n)$, then we'd be able to find a real constant $c > 0$ and an integer constant $n_0 \geq 1$ such that:

$$n^2 \leq cn, \text{ for all } n \geq n_0$$

But if we divide both sides by n (we can always assume that n is greater than 0 because it represents a size of an input to an algorithm), we get:

$$n \leq c, \text{ for all } n \geq n_0$$

But this is impossible: c is a fixed constant, and n is a variable that can grow arbitrarily large. No matter what value we fix c to be, we can always find an n that is greater than c . Therefore, n^2 is *not* $O(n)$.

Example. Using the definition of O-notation, prove that $n^4 + n^3 + n^2 + n + 1$ is $O(n^4)$.

To do this, we need to find a real constant $c > 0$ and an integer constant $n_0 \geq 1$ such that:

$$n^4 + n^3 + n^2 + n + 1 \leq cn^4, \text{ for all } n \geq n_0$$

Dividing both sides by n^4 , we get:

$$1 + \frac{1}{n} + \frac{1}{n^2} + \frac{1}{n^3} + \frac{1}{n^4} \leq c, \text{ for all } n \geq n_0$$

As n gets big, the left-hand side of the inequality gets closer and closer to 1, and so the inequality is true if we choose c and n_0 big enough, e.g. we could set $c = 10$ and $n_0 = 100$.

Example. Using the definition of O-notation, prove that n^{k+1} is *not* $O(n^k)$ for any constant $k \geq 1$.

To do this, let's assume that it was true that n^{k+1} is $O(n^k)$ and derive a contradiction. If n^{k+1} were $O(n^k)$, then we'd be able to find a real constant $c > 0$ and an integer constant $n_0 \geq 1$ such that:

$$n^{k+1} \leq cn^k, \text{ for all } n \geq n_0$$

Dividing both sides by n^k , we get:

$$n \leq c, \text{ for all } n \geq n_0$$

It's not possible to bound the variable n by a fixed constant c , and n^{k+1} is *not* $O(n^k)$.

Example (proposition 4.9 of the textbook). Consider a polynomial of degree k , $f(n) = a_k n^k + a_{k-1} n^{k-1} + \dots + a_1 n + a_0$, where $a_k > 0$. Prove that $f(n)$ is $O(n^k)$, i.e. that a polynomial of degree k is $O(n^k)$.

To do this, we need to find a real constant $c > 0$ and an integer constant $n_0 \geq 1$ such that:

$$a_k n^k + a_{k-1} n^{k-1} + \dots + a_1 n + a_0 \leq c n^k, \text{ for all } n \geq n_0$$

Dividing both sides by n^k , we get:

$$a_k + a_{k-1} \frac{1}{n} + \dots + a_1 \frac{1}{n^{k-1}} + a_0 \frac{1}{n^k} \leq c, \text{ for all } n \geq n_0$$

As n gets big, the left hand side of the inequality gets closer and closer to a_k , and so the inequality is true if we choose c and n_0 big enough, e.g. we could set $c = a_k + a_{k-1} + \dots + a_1 + a_0$ and $n_0 = 1$.

Another way to see this is to note the following fact:

$$a_k n^k + a_{k-1} n^{k-1} + \dots + a_1 n + a_0 \leq a_k n^k + a_{k-1} n^k + \dots + a_1 n^k + a_0 n^k$$

That is, if you replace each term in the polynomial with the highest-order term, you get a polynomial that is bigger than the original polynomial. If you factor out the n^k , you get:

$$(a_k + a_{k-1} + \dots + a_1 + a_0) n^k$$

This means:

$$a_k n^k + a_{k-1} n^{k-1} + \dots + a_1 n + a_0 \leq (a_k + a_{k-1} + \dots + a_1 + a_0) n^k$$

The right-hand side is a constant times n^k , which is $O(n^k)$. Since $f(n)$ is less than, or equal to this, we can conclude that $f(n)$ is $O(n^k)$.

Example We can often simplify O-notation expressions without needing to resort to the definition. Usually we want to find the tightest and simplest O-expression

for an expression, and, intuitively, we can get there by dropping low-order terms and ignoring constant factors.

For example, $4n^2 + 3n \log n + 5n + 10$ is $O(n^2)$. That's because n^2 is the highest-order term, and the constant 4 doesn't matter in O-notation.

In general, if $f(n)$ is $O(g(n))$, then $k \cdot f(n)$ is also $O(g(n))$, where $k > 0$ is a constant. You can justify this from the definition. Since $f(n)$ is $O(g(n))$, there exists a real constant $c > 0$ and an integer constant $n_0 \geq 1$ such that:

$$f(n) \leq cg(n), \text{ for all } n \geq n_0$$

Multiplying both sides by k , we get:

$$k \cdot f(n) \leq k \cdot cg(n), \text{ for all } n \geq n_0$$

This shows that $k \cdot f(n)$ is less than, or equal to, a constant times $g(n)$ for all the $n \geq n_0$, and n_0 comes from the proof for $f(n)$ being $O(g(n))$.

Example. Sometimes you can use basic algebra to simplify O-notation expressions. For example, what is the tightest O-notation description of $\log_2 2^{n+1}$? Using the rules of logarithms, we can simplify this to $(n+1) \log_2 2 = (n+1) \cdot 1 = n+1$. Therefore, $\log_2 2^{n+1}$ is $O(n)$.

From this we can also see that $\log n^2 + 35n + \log_2 2^{n+1}$ is $O(n)$. That's because $\log n^2 = 2 \log n$ is $O(\log n)$, $35n$ is $O(n)$, and $\log_2 2^{n+1}$ is $O(n)$. Since $O(n)$ is a higher-order term than $\log n$, we can ignore the $\log n^2$. This gives us an $O(n)$ term plus an $O(n)$ term, which is $O(n)$.